CHAPTER 5

#digitalfirst: A New Way of Working

Taking everything into account that we have discussed so far, we propose a new way of working for OEMs, which we are simply calling #digitalfirst.

The fluid nature of today's consumer preferences means we can't definitively predict today which features will be in demand tomorrow. However, a key certainty emerges: OEMs that can't swiftly explore, test, and deliver new features at scale may find themselves incapable of crafting the engaging customer experiences today's consumers demand. In our rapidly evolving digital age, the agility to innovate quickly and effectively is not just desirable—it's essential to staying relevant in the automotive industry.

Consequently, #digitalfirst starts with customer experiences and works backward to the technology. We have co-founded the digital.auto initiative to support this. "Digital first, auto second" is not just a tagline, but the core ethos guiding this transformation. As shown in Figure 5-1, #digitalfirst assumes that an OEM has to undergo three tectonic shifts: the shift north (of the vehicle API), the shift left (toward early-stage testing), and the shift toward virtualized development. We will look at each of these shifts in more detail.

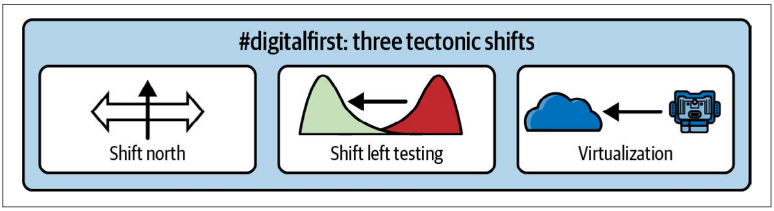

Figure 5-1. The three tectonic shifts underlying #digitalfirst

Shift North

The shift north involves moving functionalities from the safety domain to the QM domain, creating a separation between the domains via well-defined APIs. This is driven by the extra effort that will remain for every development in the safety domain—validation, homologation, and extra documentation—that is usually required to a lesser degree in the QM world.

Shifting code north into the QM world, as shown in Figure 5-2, means that modern software engineering techniques and tools can be used, speeding up development and making updates post-SOP much easier. In addition, in this domain, even software developers who don't have years of experience in the automotive sector (modern software engineers who also know ISO 26262 are rare) can work productively, thus accelerating development significantly.

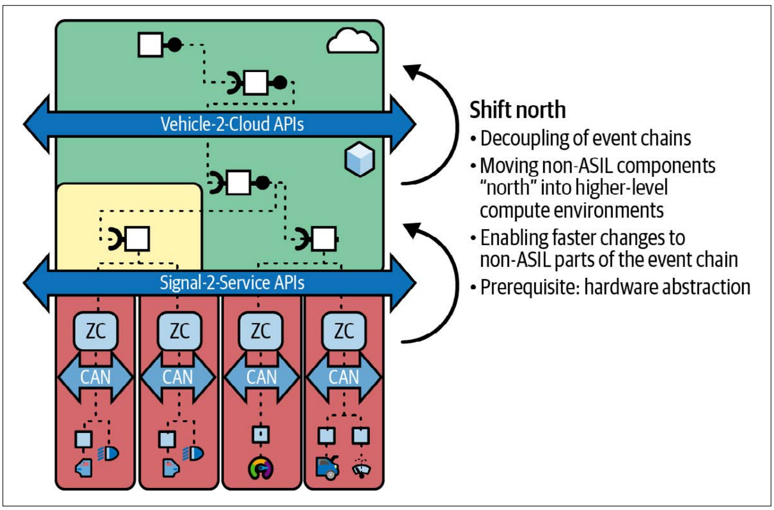

Figure 5-2. Shift north

Shift Left

The shift left refers to exploring customer features and conducting user testing as early as possible in the development process, as shown in Figure 5-3. Many of today's cars offer an abundance of buttons and features, many of which remain unknown or unused by users. Hence, the objective is to discern truly desirable features and maximize customer satisfaction through early exploration and testing.

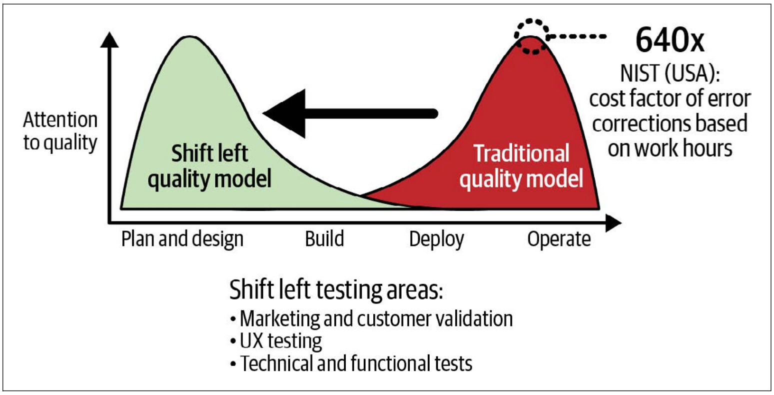

Figure 5-3. Shift-left testing

According to a report from the National Institute of Standards and Technology (NIST), an agency of the US Department of Commerce, the cost of fixing problems downstream in the development process can be up to 640 times the original development cost. And this does not factor in the cost for losing business due to unhappy customers, or customers who simply don't care about certain features that the developers were sure the customers would love but never cared to ask.

Virtualization

Virtualization emphasizes the development and testing of systems in virtual cloud environments. An interesting organization in this space is SOAFEE, which is developing virtualization concepts in the context for ARM-centric hardware architectures. The main motivation here is to overcome the traditional tight coupling between hardware and software in automotive development, where software engineers must wait for expensive, early versions of hardware for development and testing. Integration and testing of components are costly and complex due to limited prototypes and numerous vehicle variations, such as different engines, trim levels, or country-specific requirements.

The capacity that cloud environments offer for infinite scalability and cost reduction, along with the use of virtual electronic control units (vECUs) or virtualized cars, presents a solution to these challenges, as shown in Figure 5-4.

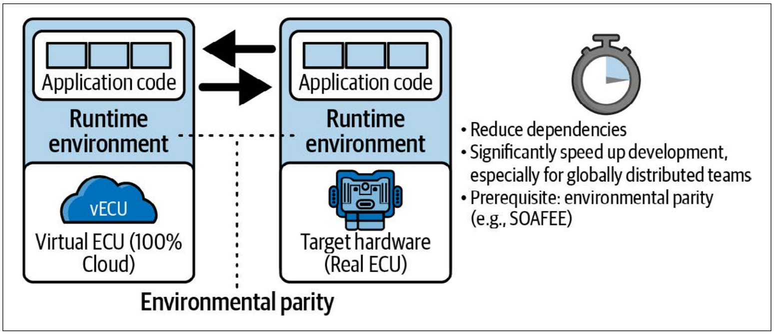

Figure 5-4. Virtualization

Summary

To deliver on the vision of the "habitat on wheels," with rich cross-domain applications and data fusion delivered 10 times as fast, OEMs must overcome significant impediments presented by functional safety requirements, technical constraints, and organizational constraints ("clash of two worlds").

The vehicle OS can be a powerful platform, enabling rapid application development in the QM world via service-oriented architectures, OTA, and vehicle app stores. Combining the SDV and AI is important for data-driven applications.

To manage "development at different speeds," OEMs must embrace VSM and use hardware abstraction and vehicle APIs to create a loose coupling, not only on the technical level, but also on the organizational level between the digital and the physical value streams, as shown in Figure 5-5.

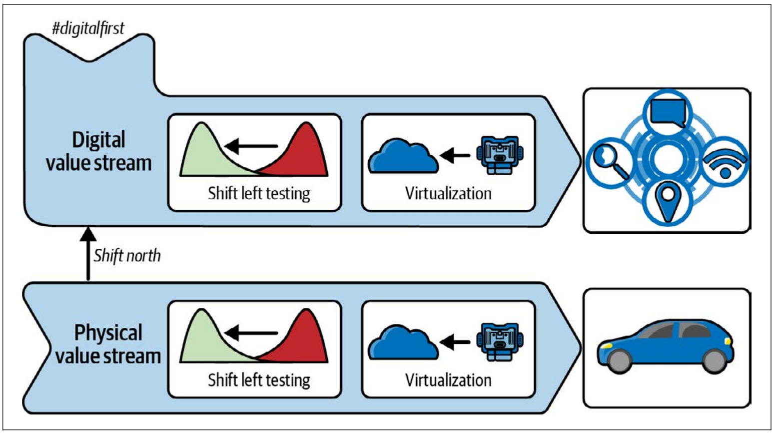

Figure 5-5. #digitalfirst and VSM for the digital.auto

#digitalfirst is a new way of working that combines three tectonic shifts:

Shift north
: Breaking up event chains and focusing on QM application development north of the vehicle API

Shift left
: Left-shifting user tests and getting early customer validation of the integrated digital/physical experience

Shift toward virtualization
: Further decoupling of hardware and software development, thus increasing agility

Together, the SDV and #digitalfirst can deliver a number of benefits for OEMs and the entire supply chain. Early and continuous user feedback helps ensure that digital investments are paying off as intended. Avoiding investments in unwanted digital features helps ensure that development capacities are used where they create the highest customer value. Significantly increasing development speed (10x) provides the agility that is required to react quickly to changing customer needs and preferences. The divide and conquer approach enabled via hardware abstraction and specialized value streams helps manage the complexity of today's automotive world. Virtualization helps reduce the cost and complexity of managing too many hardware prototypes.